



University
of Glasgow

Tuesday 3 May 2022

09:30-11:00 BST

Duration: 1 hour 30 minutes

Additional time: 30 minutes

Timed exam – fixed start time

DEGREES OF MSc, MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

TEXT AS DATA M COMPSCI 5096

(Answer all 3 questions)

This examination paper is an open book, online assessment and is worth a total of 51 marks.

1. **Question on Distributional Semantics and Word Embedding. (Total marks: 18)**

Consider the problem of finding an outlier word from among a list of other similar words, e.g., out of the following set of words -

linux, windows, solaris, android, java,

the word 'java' is an outlier (because the other words are names of operating systems).

Given a list of such words your task is to automatically infer the outlier word. With respect to this task, answer the following questions.

- (a) An approach to solve the word intrusion problem is to represent words as vectors and then make use of the relative distances/similarities between the vectors for finding the outlier.

Assuming you know (by the output of some process) the vector $\mathbf{w}$ for a word w , describe the pseudo-code of finding the outlier word.

Task: Describe the pseudo-code of this algorithm. Clearly state your assumptions and introduce your notations in the algorithm.

[5]

Example Answer: The approach described compares each word vector with the centroid of the other word vectors. The word with the lowest similarity to the other vectors is deemed the outlier.

- For each $i = 1 \dots n$ (n being the number of given words), assume the i^{th} word as the candidate outlier, $c = w_i$.
 - Let us denote the rest of the words as $R = V - \{c\}$ (V is the set $w_1, \dots, w_n$.)
 - Compute the centre of the set V as $\mu_i = \frac{1}{|R|} \sum_{v \in R} \mathbf{v}$.
 - Compute the similarity (e.g. cosine) of c with that of μ_i , i.e., $s_i = \frac{\mu_i \cdot \mathbf{c}}{|\mu_i| |\mathbf{c}|}$.
- Output the word w_i with the smallest s_i .

One solution to the word outlier detection problem that does not require learning any parameters (via gradient descent) is the distributional semantics vector approach, where each word is represented via a bag-of-words vector of contexts. Now, **answer the following:**

- (b) The window size, k , used to define the contexts for each word is an important parameter of this approach. What happens if k is too large or too small? [2]

Example Answer: Too large k means that the contexts are too broad and noisy, as a result of which the vectors would be inconsistent. Too small a context means that the contexts won't have enough information to represent the words.

- (c) Describe the pseudo-code of this approach that requires only a single pass through a collection (clearly describe the data structures for an efficient solution). [5]

Example Answer:

- k is the window size
- Initialise D as an (empty) dictionary of sparse vectors
- For each $i = 1 \dots P$ (where P is the total number of tokens),
 - $window \leftarrow [t_{i-k}, \dots, t_{i-1}, \underbrace{t_i}, t_{i+1}, \dots, t_{i+k}]$
 - $v \leftarrow \text{vectorize_with_term_frequency}(window)$
 - Add v to the current vector for the token t_i in D

- (d) Discuss (with an example) why the vectors of function words (frequent words, such as ‘of’, ‘the’ etc.) obtained with this approach are not reliable. [2]

Example Answer: The vectors of function words (frequent words, such as ‘of’, ‘the’ etc.) are likely to be unreliable because the contexts around these words are inconsistent, i.e., given a word w the values of $P(c_i|w)$ will be close to a uniform distribution (hence an uncertainty in associating a word to its context).

Now consider word2vec, which is a noise contrastive estimation based method that *learns* the vectors for each word. With respect to word2vec, **answer the following questions**.

- (e) What is the role of negative samples in the objective function of word2vec? [2]

Example Answer: The role of negative samples in word2vec is that it provides additional examples for reference outputs which makes it possible to learn the parameters with instances from two classes instead of just 1.

- (f) Comment about word2vec’s output for a word with multiple meanings, such as `jaguar`, `bank` or `python`. What would you expect to find as the nearest neighbors of such polysemous words? What is the problem if you use such vectors from such words for another task such as text classification? [2]

Example Answer: The nearest neighbors of such words will correspond to words related to the **multiple senses** of the term, e.g., the nearest neighbor (NN) set of the word ‘python’ could contain words such as `language`, `program`, `numpy` (related

to the ‘programming language’ sense), and words such as `snake`, `skin`, `poison` related to the ‘animal’ sense of the word.

The vector being close to both the senses can result in poorer performance on a downstream task, such as text classification.

2. **Question on word frequencies and language model (Total marks: 15)**

An alien probe crashes to Earth containing a short passage of alien text. The alien text uses a five letter alphabet: [a, b, c, d, e] with no punctuation or spaces. Below is a short section of the text:

abcaedabccbaedabceda

- (a) Using character n-grams, write out all of the trigrams that appear more than once with their frequency for the sample text above. [3]

Example Answer:

trigram	frequency
abc	3
eda	3
aed	2
dab	2

- (b) Provide the theoretical maximum number of character n-grams for the alien probe full text for $n = 1, 2, 3, 4$ and 5 . The full text found in the probe is 593 characters long. [3]

Example Answer:

n	max n-grams
1	5
2	25
3	125
4	590
5	589

- (c) A linguist makes a breakthrough in understanding the tokens used in the alien text. She provides two possible ways to tokenize the sample text.

- (i) In plain English, explain a single rule that could reproduce this first tokenization

a bca eda bccba eda bceda

[1]

Example Answer: Start a new token whenever the previous character is 'a'.

- (ii) In plain English, explain a single rule that could reproduce this second tokenization

ab caedab ccbaedab ceda

[1]

Example Answer: Start a new token whenever the previous character is 'b' and the current token is 'c'

- (d) More alien probes crash land in different parts of the world. Scientists want to measure the similarity between the text found in each probe. Here are two tokenized probe texts fragments.

Probe Text A:

a	eda	bceda	eda	bceda	bce
---	-----	-------	-----	-------	-----

Probe Text B:

ca	eda	bcba	eda	bceda	eda	bce
----	-----	------	-----	-------	-----	-----

- (i) Calculate the Sørensen–Dice Coefficient and Jaccard Similarity between the two probe texts. Show your work.

[4]

Example Answer:

$$\text{sorensondice}(A, B) = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \times 3}{5 + 5} = \frac{3}{5} = 0.6$$

$$\text{jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{3}{7} \approx 0.429$$

- (ii) Calculate the similarity between the third probe text (Probe Text C below) and the two prior probe texts using the Sørensen–Dice Coefficient. Using these results, show that the Sørensen–Dice Coefficient is a semi-metric as it breaks the triangle inequality.

Probe Text C:

beda	bceda	bceca	ebeda	bceda	b
------	-------	-------	-------	-------	---

[3]

Example Answer:

$$\text{sorensondice}(A, C) = \frac{2|A \cap C|}{|A| + |C|} = \frac{2 \times 1}{5 + 5} = \frac{1}{5} = 0.2$$

$$\text{sorensondice}(B, C) = \frac{2|B \cap C|}{|B| + |C|} = \frac{2 \times 1}{5 + 5} = \frac{1}{5} = 0.2$$

The triangle inequality holds if the following is true:

$$\text{sorensondice}(A, B) \leq \text{sorensondice}(A, C) + \text{sorensondice}(B, C)$$

$$0.6 \leq 0.2 + 0.2$$

It doesn't hold so the triangle inequality is broken.

3. **This question is about Natural Language Processing (Total marks: 18)**

You just landed an awesome job at the Intellectual Property Office. As your first project, you have been tasked with automatically classifying submitted patent applications into one of the eight broad International Patent Classification sections, as shown here:

Section	Subject Matter	Section	Subject Matter
A	Human necessities	E	Civil engineering; Building accessories
B	Performing operations; Transporting	F	Mechanics; Lighting; Heating
C	Chemistry; Metallurgy	G	Instrumentation
D	Textiles; Paper	H	Electricity

- (a) You start by applying a typical pre-processing pipeline that consists of case normalisation and a stemmer. Within the context of patent classification application, clearly justify these two pre-processing stages and provide an example that shows why it could lead to improved classification performance.

[4]

Example Answer:

- **Case normalisation:** Inconsistent use of upper case characters could be present in patents based on position in the sentence. For instance, the word “textile” may be a strong indicator of Section D, but if it appears at the start of a sentence, it will appear as “Textile”; case folding resolves this issue by treating them as the same token.
- **Stemming:** Plurality and tense will often not change the topic of a patent. For instance, “textiles” and “textile” likely refer to the same thing (and probably indicate Section D); stemming resolves this issue by treating them as the same token.

- (b) You recall from Text as Data that NLP features, such as parts of speech, are often helpful for classification tasks. Within the context of patent classification, provide and justify a specific example where considering a word along with its part-of-speech may help distinguish between two of the above sections.

[3]

Example Answer:

The word “light” may appear frequently in patents from both category F (e.g., light fixtures) and D (e.g., light fabric). In F, “light” will frequently be a noun (i.e., the particle of light), while in D it will frequently be an adjective (i.e., lightweight).

- (c) Armed with the above intuition, you select an off-the-shelf part-of-speech tagger (based on a Hidden Markov Model) that reports 97% accuracy and apply it to some sample patents to ensure that it produces reasonable part-of-speech tags. To your dismay, you find that it frequently makes mistakes. On closer inspection, you observe that the errors are usually on specialised, domain-specific language in the patents. Explain why this problem arises and what you could do to fix it. [4]

Example Answer: You are observing a domain mismatch. Your domain (patent text) contains specialized language (e.g., words and sentence structures) that the off-the-shelf POS tagger did not encounter in its training data (often news). Two ways to fix this would be:

1. You could tag patents and fine-tune the off-the-shelf classifier
2. You could apply some pre- or post-processing to apply better defaults to this specialized language e.g., using an external resource like a domain-specific dictionary. This will work best when the domain-specific words only act as a single part-of-speech.

- (d) You want to identify whether two systems (called System A and System B) are better than a baseline method at the classification task. The following table shows intrinsic evaluation metrics obtained over the classification on the train and test sets:

	Train Set		Test Set	
	Precision	Recall	Precision	Recall
Baseline	0.61	0.42	0.56	0.50
System A	0.62*	0.43*	0.58*	0.51
System B	0.67*	0.48*	0.51	0.42

* statistical significance w.r.t. baseline (t -test with p -value < 0.05)

Discuss the effectiveness (e.g., generalizability, overfit/underfit, performance on training/test sets etc.) of the models A and B in comparison to the ‘Baseline’ method. [3]

Example Answer:

- System B is the best performing model on the training set
- System B performs better than baseline on the train set, but worse in the test set, which can indicate a degree of overfitting.
- System A performs better than the baseline both in the train set and test set.

- (e) Meanwhile, another team has been busy building a BERT-based text classifier, and they have found that it also works well on the task. You decide to join forces with them. Without

using an ensemble approach, how might you go about including explicit parts-of-speech into their BERT-based model? How is the technique different than the approach you took in your linear bag-of-features model? [4]

Example Answer:

In the bag-of-features model, each term-POS pair was treated as a separate feature in a sparse vector; if the word did not appear in the text as the specific POS, it was given a value of zero. In contrast, BERT models sequences of words, where each word is represented by an embedding. Therefore, to incorporate POS tags into a BERT model, you will need to augment these embeddings at the input layer.

Akin to positional or segment embeddings, a new bank of embeddings (one for each POS) could be introduced into the model. These would be added to the input embedding. (Or a similar approach of including POS embeddings within the sequence, as long as the optional information is retained.)